Computer Networks Using Sockets

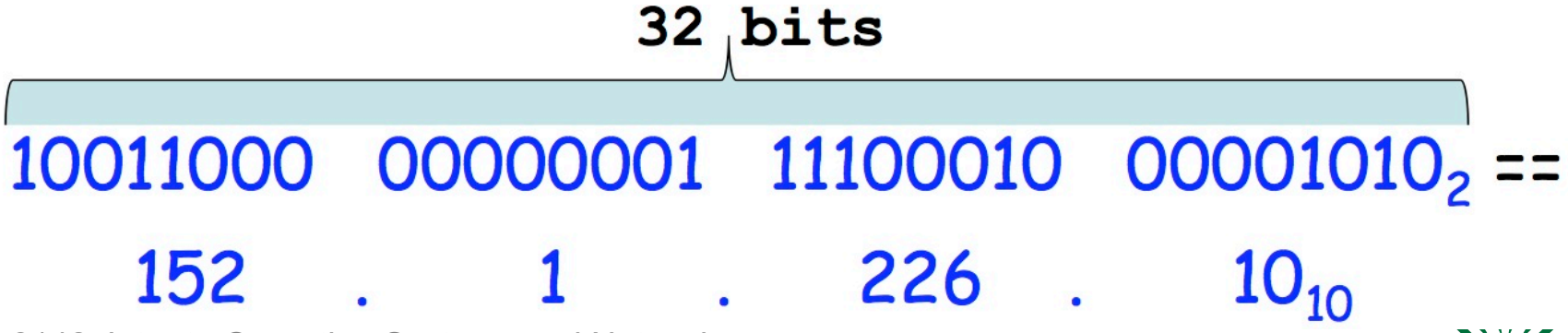
Introduction

- Distributed applications are everywhere: the web, email, internet phones, file sharing, multiplayer games, ecommerce, ...
- Most common type of application: **client/server**
  - A client process on one computer, server process on another, communicating via Internet protocols

    By far the most common programming API: **sockets**

IPv4 Addressing

- To deliver a datagram to a destination (endpoint), you must specify...
  - The **IP address** (32 bits) of the host network interface
  - The **port number** (16 bits) of the receiving application process
- A 32-bit IPv4 address is usually written in



Addressing... (cont'd)

- The port number is bound to the receiving application by a function call
- Examples:

Port	Application
80	Web
21	File transfer
25	Email
53	DNS
139	Net BIOS
513	Remote login

Domain Names

- Most people can't remember numbers; prefer to use **names** instead
- Requires software (DNS) to translate easy-to-remember names into IPv4 addresses / port numbers
  e.g., **http://www.uncc.edu** ⇔ 152.15.47.208, port 80

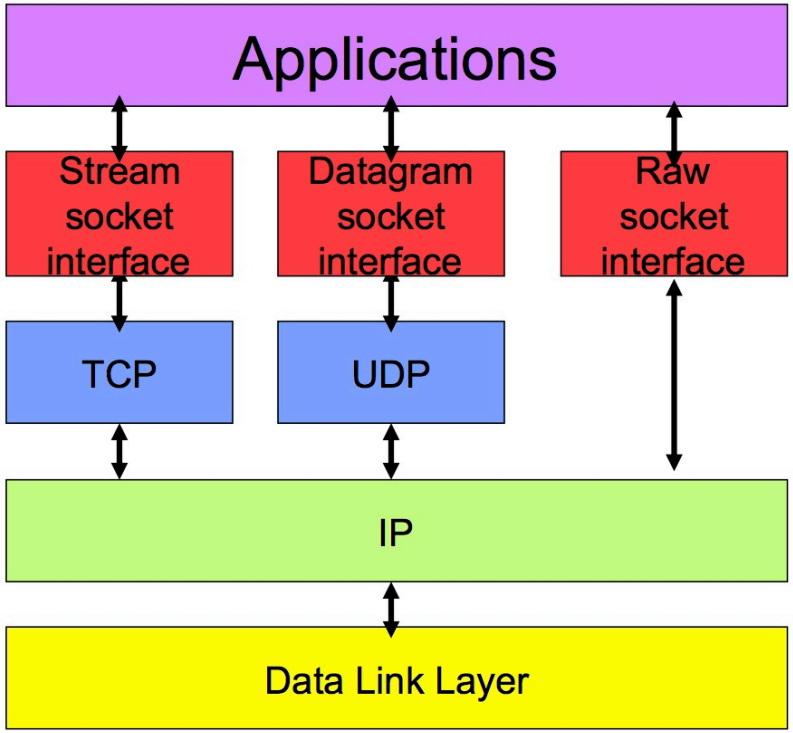
The Sockets API

- Goals: flexible, portable, easy to use
- Introduced in 1981, available today on every platform
- Types of service:
  - **datagram** (UDP)
  - **stream** (TCP)

...Sockets (cont'd)

- A **socket** is a data structure that
  - Holds addresses of two communicating endpoints
  - Maintains state of the connection
  - Provides space for buffering data during transfer
- You (the programmer) don't need to know the details; you just need the **handle** to the socket

The Network "Stack”



Constant and Function Definitions

- You'll need to include most of these **#define**'s for your programs to compile (**Linux/Unix**)

```
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <sys/types.h>
```

Data Structure: Socket Addresses

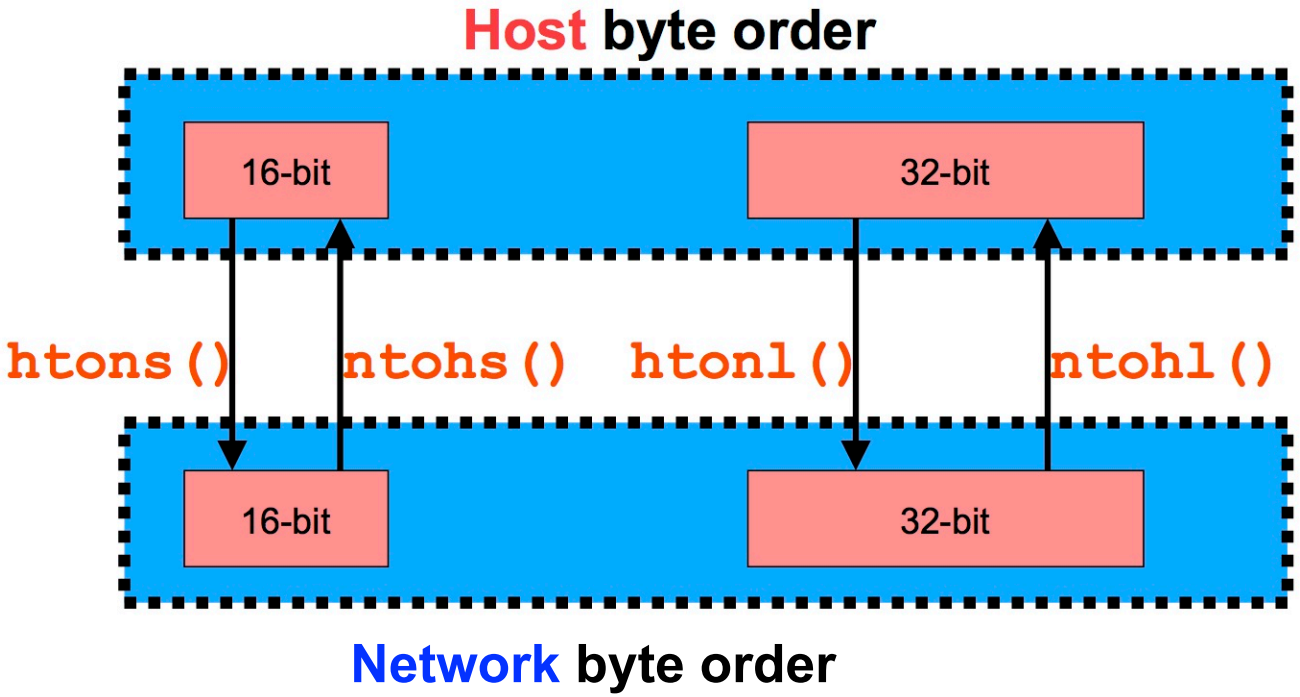
Socket address = IPv4 address + port number

```
struct in_addr {
    u_long s_addr;          /* 32-bit IPv4 addr */
};

struct sockaddr_in {
    …some stuff…
    u_short  sin_port;          /* 16 bit port */
    struct in_addr sin_addr;
    …some stuff…
};
```

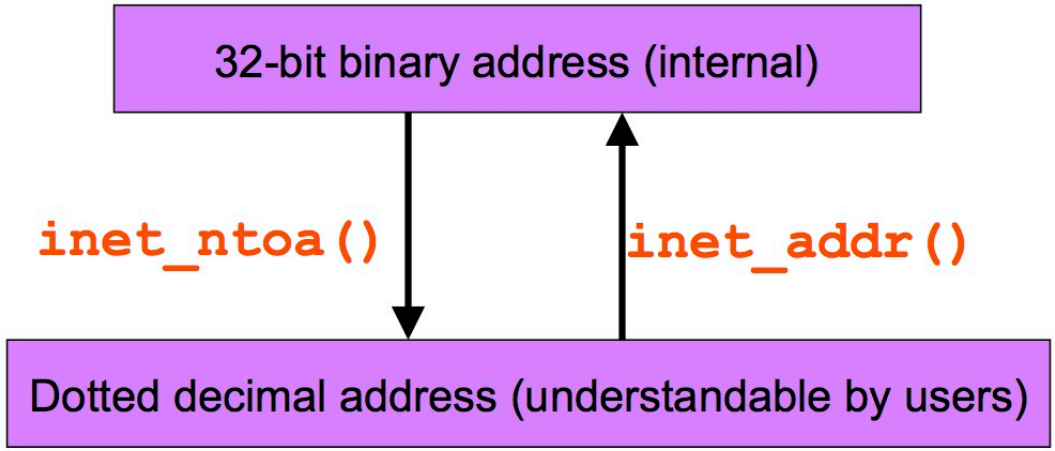
Byte-Ordering Translations

IP addresses and ports in datagrams are stored using **big-endian** ordering, hosts often use **little-endian** ordering; **translation is required**



Address Translations

Users specify addresses in dotted decimal form (string), computers need addresses in 32-bit form; another translation required.




```
int addr =  inet_addr(char *str);
char *str =  inet_ntoa(struct in_addr in);
```

Examples

From dotted decimal to 32-bit address...

```
char * dd_hostnum = "152.14.86.8";
sin.sin_family = AF_INET;
sin.sin_addr.s_addr = inet_addr(dd_hostnum);
```

Or even from host name to 32-bit address...

```
char * hostname = "www.csc.ncsu.edu";
struct hostent  *phe; /* ptr to host info */
sin.sin_family = AF_INET;
if ( phe = gethostbyname(hostname) )
    memcpy(&sin.sin_addr, phe->h_addr, ↲
                          phe->h_length);
```

The above is called **name resolving**

Translating the Port Number

```
char * transport = "tcp";
char * service = "http";
struct servent  *pse; /* ptr to service entry */

if ( pse = getservbyname(service, transport) )
    sin.sin_port = pse->s_port;
```

or

```
unsigned short servicenum = 80;

sin.sin_port = htons( servicenum );
```

Creating a Socket

Create a socket data structure, and return a handle to it

```
type = SOCK_DGRAM;  /* UDP */
-or-
type = SOCK_STREAM; /* TCP */

int s = socket(PF_INET, type, 0);
```

TCP Sockets

Client-Server Interaction: TCP

Server	Client
① Create master socket data structure for communication, and declare type `socket()`	① Create socket data structure for communication, and declare type `socket()`
② Designate receiving (local) port # and IPv4 address `bind()`	
③ Wait for incoming connection requests from clients `listen()`	

Server	Client
④ Block until incoming connection request, create a new socket for the request and record remote IPv4 address and port # `accept()`	
	⑤ Designate "remote" port # and IPv4 address (and automatically choose local port) + send connection request to server `connect()`
(`accept()` unblocks)	

Server	Client
⑥ Block until a request datagram is received from a client `recv()`	
	⑦ Send request datagram to the server `send()`
(`recv()` unblocks)	⑧ Block until response datagram(s) received from the server `recv()`
⑨ Send response datagram(s) to client and return to step #6 `send()`	

Server	Client
	(read() unblocks)
	⑩ Return to step #7 until no more requests to send
	11 Terminate the socket close()
12 Terminate the socket when closed by client and return to step #4 close()	

Step #1: Creating A Socket

Create a TCP socket for communication

```
int srvs = socket(PF_INET, SOCK_STREAM, 0);
```

```
int clnts = socket(PF_INET, SOCK_STREAM, 0);
```

Step #2: Bind Server Port/Address

- **Servers** specify a "well-known" port to use for incoming requests
- Use **INADDR_ANY** to bind the socket to **all** of the machine's IPv4 addresses

Example

```
myaddr.sin_addr.s_addr = INADDR_ANY;
myaddr.sin_port = htons(myportnum);
int result = bind(srvs, ↲
    (struct sockaddr *) &myaddr, sizeof(myaddr));
```

Step #3: Wait for Connection (server)

- Places a socket in **passive** mode
  - Ready to accept incoming connection requests from anyone
- Allows up to **backlog pending** connections to be waiting for an **accept()** and not discarded

```
int result = listen(int s, int backlog);
```

Step #4: Accept Connection Request (server)

- Blocks server process until a connection request arrives
- Then…
  - Removes the next connection request from queue
  - Creates a new socket (for this specific client)
  - Binds remote address (from connection request) to socket

Step #5: Connecting To Server (client)

- Establishes connection (**active** open) to server by datagram exchange
- Binds server address as remote endpoint
- Chooses a local port number
- May fail! Reasons?
  - Host is down
  - Host does not have active service bound to port

```
int result = connect(int clients, ↲
   struct sockaddr *srvaddr, int srvaddrlen);
```

Sending Data (both)

- Flags control transmission
  - e.g., specify “urgent” data
- As long as connection is established, **all** data will be delivered (**guaranteed**)

```
int   send(int s, char* buf, ↲
           int buflen, int flags);
```

Steps #7, #9: Closing A Connection

```
int  close(int s);
```

- Actions
  - Decrements reference count for socket
  - Deallocates socket when reference count = 0
  - Any unread data from the other end will be discarded
- Problem: how do you know if the **other end** of connection is ready to terminate?

Partially Closing A Connection

```
int result = shutdown(int s, int direction);
```

- **direction** = 0: close the input (reading) end
  - "I'm not listening anymore, but I have something more to say..." ?!
- **direction** = 1: close the output (writing) end
  - "I have nothing more to say, but I'm still listening for your response..."

Receiving Data (both)

```
int result = recv(int s, char* buf, ↲
                  int buflen, int flags);
```

- Flags control reception, see documentation for details...
- Reading is **stream-oriented**
  - Data may be read a few bytes at a time, an entire datagram at a time, etc.
- If other end closed the connection, and no more data to read, **read()** returns **0 to indicate EOF**

TCP Receiving Data

- Data delivery is stream-oriented; there are no record, or request, boundaries!
- Easy to keep looping, reading all the input datagrams for one request…

```
char *bp = bufin;
int max = MAXBUF;
while ( (n = recv (s, bp, max, 0)) > 0 ) {
    bp += n;
    max = MAXBUF - (bp - bufin);
}
*bp = '\0';
/* data in bufin at this point */
```

TCP Receiving... (cont'd)

- How can the server tell when the client has no more requests if there are no boundaries?
- The easy (common) way
  1. Client sends one request, half closes the connection
  2. Server recvs request (>= 1 datagrams), detects close
  3. Server sends one response, completes close of the connection
  4. Client recvs response (>= 1 datagrams), detects close by server
  - Each time the client has a new request, a new connection to the server is established